

Kreacijski paterni – Earn&Learn

1. Singleton

Osigurava da klasa ima samo jednu instancu u cijelom sistemu i pruža jedinstvenu, globalnu tačku pristupa toj instanci.

Korištenje: U aplikaciji *Earn&Learn*, koja radi sa novčanim transakcijama i zakazivanjem termina, veoma je važno imati jedno sigurno mjesto za praćenje grešaka i radnji u sistemu. Zato uvodimo klasu *Logger* kao Singleton. Kada bi različiti dijelovi aplikacije (npr. modul za plaćanje i modul za korisnike) sami kreirali svoje logger-e, dolazilo bi do problema jer bi svi pokušavali istovremeno pisati u isti fajl na disku. To bi usporilo aplikaciju i pomiješalo zapise. Singleton osigurava da cijeli sistem koristi samo jednu instancu koja uredno i bezbjedno zapisuje sve podatke.

- Klasa je izolovana na dijagramu i definisana kao centralizovani uslužni servis na nivou aplikacije namijenjen za reviziju sistemskih aktivnosti.
-

2. Factory Method

Definiše interfejs za kreiranje objekata, ali prepušta podklasama ili specijalizovanim metodama odluku o tome koju će konkretnu klasu instancirati.

Korištenje: Naša klasa *Korisnik* preko atributa *Uloga* razlikuje tri tipa korisnika: *Studenta*, *Tutora* i *Administratora*. Ako ne bismo koristili ovaj obrazac, morali bismo na svakom mjestu u kodu gdje se vrši registracija pisati dugačke if-else ili switch-case uslove kako bismo podesili specifična polja (npr. cijenu po satu za *Tutora* ili broj indeksa za *Studenta*). Klasa *KorisnikFactory* rješava ovaj problem tako što skuplja svu tu logiku na jedno mjesto. Mi fabрики samo prosljedimo ulogu, a ona nam vrši provjeru i vraća ispravno napravljen objekat korisnika. To čini kod čistijim i lakšim za proširivanje.

3. Builder

Razdvaja proces konstrukcije složenog objekta od njegove reprezentacije, omogućavajući kreiranje objekta korak po korak.

Korištenje: Klasa *Termin* je najkomplikovanija klasa u našem sistemu jer sadrži mnogo podataka: datum i vrijeme, tip instrukcija (*Online* ili *Uživo*), status termina, veze prema studentu i tutoru, te QR kod za potvrdu časa. Klasični konstruktori sa ovoliko puno parametara su nepregledni i lako se može pogriješiti pri slanju podataka. Uvođenjem klase *TerminBuilder*, omogućeno je da se termin sklapa postepeno, dio po dio. Na samom kraju, kroz metodu *.build()*, sistem automatski generiše i QR kod, što garantuje da se ne može kreirati nekompletan ili nevalidan termin.

4. Abstract Factory – Nije korišten

Omogućava kreiranje porodica povezanih ili zavisnih objekata bez specificiranja njihovih konkretnih klasa.

Ovaj uzorak se koristi kada aplikacija mora raditi sa više različitih grupa proizvoda (na primjer, ako aplikacija treba imati različite stilove dugmadi i prozora za Windows, a različite za Mac). Pošto su entiteti u našoj aplikaciji *Earn&Learn* (kao što su Transakcija i Recenzija) jednostavni, jedinstveni i nemaju nikakve paralelne varijacije u zavisnosti od operativnog sistema ili platforme, ovaj uzorak nam nije potreban. Njegovo dodavanje bi samo stvorilo gomilu suvišnih klasa i zakomplikovalo projekat bez ikakve koristi.

5. Prototype – Nije korišten

Omogućava kreiranje novih objekata kopiranjem (kloniranjem) postojećih instanci, umjesto kreiranjem ispočetka kroz konstruktor.

U Prototype uzorak je koristan kada je pravljenje novog objekta previše "skupo" za procesor (npr. kada objekat zahtijeva komplikovane matematičke proračune ili dugotrajno povlačenje masovnih podataka iz fajlova). U našem sistemu to nije slučaj. Svi naši entiteti (Korisnik, Termin, Predmet) kreiraju se tako što sam korisnik unese svježe podatke kroz forme na ekranu. Svaki objekat dobija svoj jedinstveni ID u bazi podataka i potpuno je unikatna cjelina, zbog čega nemamo nikakvu potrebu za kloniranjem postojećih objekata.